RankCloak Measures Recovery and Detectability of Synthetic Cryptographic Artifacts Encoded in Language Model Text

Alexander V. Mantzaris[1,*]

[1]School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL, United States of America
[*]Correspondence: alexander.mantzaris@ucf.edu

ABSTRACT

RankCloak is a measurement and evaluation framework that encodes synthetic cryptographic artifacts through language-model next-token ranks and realizes those ranks as generated text. We compare direct subword representation with bounded ASCII-byte and hex-nibble codecs, including segmented forced spans followed by non-payload tails. The completed primary evaluation used 480 public deterministic artifacts, three open-source model families, and 18 English prompt templates. All 6,480 payload trials recovered under saved-token exact-copy replay, but this endpoint requires the recorded token IDs, model, tokenizer, quantization, prompt rendering, filters, and span boundaries. Detokenizing and retokenizing visible text recovered only 88/144 in the bounded robustness sample; markdown transfer, paraphrase, and cross-model decoding recovered none. After strict component-level deduplication, matched ROC-AUC was 0.9868 for TextCNN, 0.9994 for DeBERTa-v3-base, and 0.999995 for an exact-model-aware surprisal attacker. Secondary human-authored controls and leave-one-family-out tests did not remove the adverse detectability finding. Strict entropy gating reduced text-only ROC-AUC in the bounded study, but capacity fell to 4.0513 bits/token conditional on 114 completed cases, mean length increased 2.7615-fold, six cases failed to complete, and model-aware ROC-AUC remained 0.9904. For one Qwen revision, saved-ID recovery was 960/960 under both Q4_K_M and Q8_0, while visible-text recovery was 732/960 and 758/960; the paired difference was not statistically supported. RankCloak therefore measures recovery, capacity, cover cost, and detectability; it does not demonstrate an undetectable, robust, secure, deployable, or practical covert channel.

Introduction

Generative language models provide a new medium for text steganography because a prompt can define a next-token distribution and a topic direction in which generated text appears. In the Calgacus rank-transcoding construction[1], a sender tokenizes an arbitrary hidden text, records each token's context-dependent rank, and generates a cover text by selecting the correspondingly ranked tokens under a shared cover prompt; the receiver reverses these steps under the same contexts. Calgacus thereby transcodes one hidden-text token into one cover-text token. RankCloak retains that rank-transfer mechanism but also maps exact serialized cryptographic artifacts through direct-subword and bounded codecs, for which payload and cover token counts need not equal the Calgacus baseline. This mechanism connects classical work on covert channels, information-theoretic steganography, information hiding, and provably secure steganography[2–5] with neural and LLM-based linguistic steganography, where hidden information is embedded through choices made during language generation[6–14]. Related work on LLM watermarking also treats token selection as a carrier of hidden signals[15]. Cai et al. use an entropy threshold to govern watermark insertion[16]; their receiver detects whether output is marked, whereas payload-carrying steganography must recover an arbitrary message. The present entropy-gated RankCloak experiment adopts the selective-position motivation, not the watermarking task or its guarantees.

A long-running goal in this area is to make arbitrary, encrypted, or otherwise high-entropy data appear as ordinary communication. Early mimic-function work used statistical source models and inverse compression ideas to generate outputs with distributional properties similar to a target source[17,18]. Hiding the Hidden and related linguistic-steganography systems transformed ciphertext into innocuous natural-language text that could later be transformed back into the original ciphertext[19,20]. Other text methods carried bits through synonym choice or contextual word substitution[21], while network-camouflage systems generalized a similar objective to protocol formats by making encrypted traffic resemble allowed or innocuous traffic[22,23]. RankCloak follows this same broad goal in an LLM setting. Instead of using a fixed grammar, synonym table, or protocol template, it uses the token choices made by a shared language model. The payload is first converted into ranks, and the cover prompt determines which English tokens occupy those ranks at each step. This also makes the method useful as a measurement tool since it exposes how a hidden artifact changes rank pressure, generated text quality, and the traces available for later detection.

This measurement focus also has defensive value for detecting the presence of such hidden encodings. If hashes, nonces, UUID-like values, or ciphertext strings are carried by generated tokens rather than displayed in their original syntax, surface scanners cannot rely only on visible hexadecimal, base64, UUID, or credential-shaped patterns. Detection must therefore look for indirect evidence, such as low-probability forced tokens, tokenizer artifacts, formatting fragments, unusually regular token choices, segmentation patterns, or differences between payload-bearing spans and non-payload continuations. Covert-channel and stegomalware surveys emphasize that hiding the existence of communication creates detection, capacity-limitation, and countermeasure problems across text and network media[24,25]. Text steganalysis and generated-text detection work similarly shows that statistical, neural, graph-based, and probability-based features can reveal some generated or steganographic text, while also showing that robust detection is setting-dependent and vulnerable to distribution shift or paraphrase[26–31]. The same issue is relevant to AI-safety monitoring, where hidden message passing and secret collusion among agents have been studied as ways that generated text might evade oversight[32,33]. RankCloak therefore treats hiding and detection as two sides of the same measurement problem. A useful methodology should recover payloads under declared exact-copy conditions while also producing transparent features, controls, and failure modes for steganalysis.

RankCloak proposes a measurement methodology for synthetic cryptographic artifacts encoded in generated English text. The payload classes resemble values commonly encountered in security, including hashes, authentication tags, nonces, identifiers, ciphertexts, and signatures[34–36]. These strings are challenging payloads for rank transcoding because they are not natural-language messages and do not display high probability sequences. Direct subword tokenization can represent them compactly, but the resulting payload tokens may occupy low-probability positions in the model's next-token distribution[37,38]. A cover generator forced to select such tokens can preserve exact recoverability under matched replay while producing public text with visible artifacts or unusual statistical features. The main methodological development of this paper is that cryptographic-artifact payloads can be treated as representations to be measured, not merely as strings to be passed through an LLM tokenizer. RankCloak therefore compares direct subword rank measurement with bounded-rank payload codecs, including ASCII-byte fixed-radix encodings and a hex-nibble encoding for lowercase hex-like payloads. This design makes the capacity to quality trade-off explicit. Direct subword ranks can be short but may impose large rank pressure, whereas bounded encodings constrain the allowed rank range while increasing the number of generated cover tokens. This representation view also separates payload recovery from public-message quality. RankCloak keeps the cover-side model, tokenizer, prompt family, deterministic rank-ordering rule, and replay procedure fixed while varying payload-side encodings and protocol variants. Non-segmented variants emit one cover token per payload rank under a single prompt context. Segmented variants split a payload into short rank chunks, emit payload-bearing forced spans across multiple public messages, and append non-payload natural tails after those forced spans. Because tails do not carry payload ranks, RankCloak reports forced-span and full-message quality proxies separately.

The resulting evaluation introduces RankCloak as a reproducible methodology for measuring how synthetic cryptographic artifacts can be represented as bounded rank sequences and expressed as LLM-generated text under exact-copy replay. The study is organized around how artifact-like payloads affect rank pressure, how bounded encodings change capacity and cover length, and how segmentation and natural tails change the relationship between payload-bearing spans and full messages. Exact-copy replay remains the supported recovery condition, so the sender and receiver must share the same model artifact, tokenizer, quantization, prompt rendering, filtering, deterministic configuration, codec, boundaries, and generated token identifiers. This condition is consistent with recent work showing that tokenization and retokenization mismatches can disrupt LLM-based steganography and watermarking[39]. Visible-text detokenization and retokenization is a separate, weaker recovery condition. The targeted extension evaluates a human-authored secondary control corpus, strictly deduplicated detector partitions, unseen-family and exact-model-aware attackers, validation-frozen low-FPR thresholds, entropy-gated embedding, and paired Q4_K_M/Q8_0 recovery and detector transfer. These bounded evaluations refine the method's failure surface without establishing practical robustness, security, undetectability, human-perceived naturalness, or ordinary visible-text communication.

Methods

RankCloak measures whether synthetic artifacts that do not normally look like language can be represented as LLM token ranks and then expressed as generated cover text while remaining exactly recoverable under a declared replay contract. The motivating examples are hashes, authentication tags, nonces, hexadecimal tokens, UUID identifiers, authenticated-encryption outputs, and signatures. In their original visible forms, many of these strings are recognizable by simple pattern matching (even by humans), but RankCloak asks whether the same serialized artifact can instead be carried by the token choices of an LLM-generated message. Conceptually, the method proceeds in four steps. First, the payload is converted into a sequence of integer ranks. Second, a cover prompt defines the next-token distribution used for public-text generation. Third, the sender emits the token at each required rank under that cover context. Fourth, the receiver replays the same cover context and recovers the rank of each exact saved token, following the rank-transcoding idea described by Norelli and Bronstein[1]. If the recovered rank sequence matches the encoded payload ranks and the codec reconstructs the declared serialized bytes, the payload has been decoded exactly. The algorithms below implement two RankCloak procedures, a non-segmented procedure that maps one payload into one cover sequence and decodes the complete generated sequence, and a segmented procedure that maps a hex-like payload into several short payload-bearing spans, appends non-payload natural tails, and decodes only the forced spans. These procedures support both construction-side and detection-side measurement because they provide exact-copy recovery tests, rank-pressure measurements, cover-quality proxies, labeled cover examples, and rank-derived features that can be audited by steganalysis methods.

Sender and receiver are assumed to share a common overall LLM configuration, denoted as K_{common}. This configuration includes the exact model artifact, tokenizer, quantization and backend settings, prompt rendering, deterministic rank-ordering rule, payload codec, segmentation rule where used, tail and lead-in policies where used, token filter where used, forced-span boundaries where used, and saved payload-bearing token IDs. Protocol labels select predeclared variants of this shared configuration. They are not modeled as cryptographic keys, authentication tags, signatures, or operational commands.

Recovery is evaluated under an exact-copy replay requirement for deterministic context reproduction. This means that the receiver consumes the saved generated token IDs and replays the same model, tokenizer, prompt configuration, rank-ordering rule, codec, filter, and segmentation metadata. Visible-text detokenization and retokenization is a separate, weaker replay condition. Edits, paraphrases, whitespace normalization, moderation rewrites, truncation of payload-bearing tokens, retokenization, model changes, quantization changes, prompt changes, or filter changes can alter ranks and are outside the supported exact-copy condition.

The exact-copy assumptions used throughout the experiments are:

- Shared configuration: sender and receiver use the same model artifact, tokenizer, quantization and backend, prompt rendering, rank rule, codec, filters, segmentation policy, and tail or lead-in policy.
- Deterministic rank ordering: ranks are one-indexed and computed from descending logits with token-ID tie-breaking.
- Payload codecs: ASCII-byte fixed-radix coding applies to exact serialized bytes; hex-nibble coding applies only to canonical lowercase hexadecimal payloads.
- Non-segmented decoding: the receiver replays the complete saved cover-token sequence.
- Segmented decoding: the receiver decodes only the saved payload-bearing forced spans; natural tails are public text but carry no payload ranks.
- Endpoint validation: rank recovery, representation recovery, serialized byte length, and SHA-256 identity are recorded separately, with exact serialized-payload equality as the primary endpoint.

The reader-facing protocol labels distinguish non-segmented variants, where recovery replays the complete generated cover-token sequence, from segmented variants, where recovery decodes only the payload-bearing forced span and ignores non-payload tails.

Basic rank definition

Rank-transcoding uses the rank of a token in an LLM next-token ordering as the transferable object. A rank is defined by sorting candidate tokens under a model context. Exact recovery is possible only when the sender and receiver replay the same model, tokenizer, prompt, rank rule, and generated tokens. Let M denote an autoregressive language model with vocabulary $\mathscr{V}$. For a token context h, let $z_M(v \mid h)$ be the next-token logit for token $v \in \mathscr{V}$, and let $\mathrm{id}(v)$ be its deterministic token id. The model probability is:

$$p_M(v \mid h) = \frac{\exp z_M(v \mid h)}{\sum_{u \in \mathscr{V}} \exp z_M(u \mid h)}. \tag{1}$$

Ranks are one-indexed; token ID breaks ties deterministically:

$$u \prec_h v \iff z_M(u \mid h) > z_M(v \mid h) \text{ or } \left[z_M(u \mid h) = z_M(v \mid h) \text{ and } \mathrm{id}(u) < \mathrm{id}(v)\right]. \tag{2}$$

The rank of a token and the inverse rank selector are,

$$\rho_h(v) = 1 + |\{u \in \mathscr{V} : u \prec_h v\}|, \qquad \mathrm{token}_h(r) = v \text{ such that } \rho_h(v) = r. \tag{3}$$

Here $\mathrm{token}_h(r)$ means the token at rank r in the deterministic ordering for context h. Because softmax is order-preserving, the released implementation computes this order directly from 64-bit logit values and breaks exact ties by increasing token ID.

Calgacus rank-transcoding baseline

The starting point is the Calgacus rank-transcoding construction, in which ranks rather than token identities are transferred between contexts[1]. In the Calgacus procedure, a hidden text can be tokenized as $e_1, e_2, \ldots$. For each hidden-text token e_i, the sender computes its rank under a payload-side context g_i, where g_i is determined by the preceding hidden-text tokens and any payload-side conditioning used by the protocol. The sender then emits a cover token by selecting the token with the same rank under a cover-side context h_i, where h_i is determined by the cover prompt or key and the previously emitted cover tokens.

For hidden-text token e_i, payload-side context g_i, cover-side context h_i, emitted cover token y_i, and transferred rank r_i, the baseline rank-transcoding step is:

$$r_i = \rho_{g_i}(e_i), \qquad y_i = \mathrm{token}_{h_i}(r_i), \qquad \hat{r}_i = \rho_{h_i}(y_i). \tag{4}$$

Under exact replay with the same model, tokenizer, prompt configuration, and deterministic rank rule, the recovered rank satisfies $\hat{r}_i = r_i$. The receiver can then replay the payload-side context and reconstruct the hidden-text token as $\hat{e}_i = \mathrm{token}_{g_i}(\hat{r}_i)$. This is a same-token-length arbitrary-text baseline; one hidden-text token determines one cover token. The construction shows that a rank sequence can be carried by generated text without directly emitting the hidden tokens. Direct application to cryptographic-artifact strings, however, can impose high rank pressure. Such strings are often poorly predicted by a language model, so their payload-side tokens can occupy low-probability, high-rank positions, and selecting those same ranks under a cover prompt can degrade model-likelihood and surface diagnostics even when recovery remains exact.

RankCloak keeps the cover-side rank-transfer idea from Calgacus but changes the payload-side representation. Instead of relying only on direct LLM tokenization of the displayed artifact, RankCloak maps serialized artifact strings into explicit bounded rank sequences. In the RankCloak notation used below, the displayed artifact is x, the payload codec is C, and the encoded rank sequence is $R = C(x)$. Recovery therefore proceeds by reconstructing $\hat{R}$ from the cover tokens and decoding $\hat{x} = C^{-1}(\hat{R})$, rather than by replaying a natural-language hidden-text stream token by token. The direct-subword condition retains the Calgacus payload-side ranks as a separate measurement arm. The contribution relative to the baseline is the measurement framework around artifact-specific representations: direct subword rank diagnostics, bounded ASCII-byte and hex-nibble codecs, deterministic token filtering, segmented forced spans, non-payload tails, and separate forced-span versus full-message metrics.

RankCloak bounded payload-codec methods

RankCloak changes the payload side by applying explicit codecs to displayed synthetic payload strings. The goal is to avoid treating a hash-like or ciphertext serialization as an ordinary natural-language token sequence. Instead, the displayed artifact is converted into a controlled sequence of ranks before cover generation begins. This makes rank pressure measurable by design. A codec can trade cover length against rank magnitude so that smaller rank alphabets constrain forced token choices but require more generated tokens, while more compact representations can reduce length but may impose larger rank pressure. For payload string x, codec C, encoded rank sequence R, and recovered rank sequence $\hat{R}$,

$$R = C(x) = (r_1, \ldots, r_n), \qquad \hat{x} = C^{-1}(\hat{R}). \tag{5}$$

The direct-subword arm is not folded into Eq. (5); it tokenizes the literal serialization under the Calgacus payload context and retains the resulting ranks without imposing a fixed rank alphabet. The released implementation disables special-token insertion and accepts only a reversible tokenization whose detokenization differs from the literal UTF-8 payload by an explicitly recorded leading-space prefix. Any other affix or payload-byte change makes that representation unavailable before generation; complete framing metadata appears in Supplementary Note S1.

Non-segmented cover generation and replay recovery then use the same rank selector as the Calgacus baseline, but the ranks come from the payload codec rather than only from direct payload-side LLM token ranks:

$$y_i = \mathrm{token}_{h_i}(r_i), \qquad \hat{r}_i = \rho_{h_i}(y_i), \qquad \hat{R} = (\hat{r}_1, \ldots, \hat{r}_n). \tag{6}$$

Codec-level exact recovery requires both rank recovery and an invertible codec for the displayed payload:

$$\hat{x} = x \quad \text{when} \quad \hat{R} = R \quad \text{and} \quad C^{-1}(C(x)) = x. \tag{7}$$

Hex-nibble coding applies to canonical lowercase hexadecimal payloads. It maps each displayed hexadecimal character to one bounded rank:

$$\texttt{0} \mapsto 1, \quad \texttt{1} \mapsto 2, \quad \ldots, \quad \texttt{9} \mapsto 10, \quad \texttt{a} \mapsto 11, \quad \ldots, \quad \texttt{f} \mapsto 16. \tag{8}$$

Thus each lowercase hex character maps to one rank in $1, \ldots, 16$. A 64-character SHA-256 hexadecimal serialization becomes 64 ranks under this codec. Base64 and hyphenated UUID display forms are rejected rather than silently normalized. The bounded alphabet controls maximum requested rank while increasing the number of forced positions; its relationship to observed cover diagnostics is evaluated rather than assumed. ASCII-byte fixed-radix coding applies to arbitrary displayed payload strings. It first expresses the displayed UTF-8 byte string as base-B digits d_i, then maps each digit to rank d_i+1,

$$d_i \in \{0,\ldots,B-1\}, \qquad r_i = d_i + 1. \tag{9}$$

The released codec concatenates the complete serialized byte bitstream, appends zero bits only at its end to reach a multiple of $\log_2 B$, and records the original byte length and terminal-padding count. Decoding subtracts one from each rank, validates the digit range, removes only that recorded padding, and reconstructs exactly the declared number of bytes. Consequently, the $B=8$ condition uses $\lceil 8m/3 \rceil$ ranks for m serialized bytes rather than independently padding every byte; the $B=16$ condition uses two ranks per byte. Supplementary Note S1 gives the complete metadata and edge cases.

When a deterministic token filter is configured, ranks are computed within the allowed token set A_h:

$$\rho_h^F(v) = 1 + |\{u \in A_h : u \prec_h v\}|, \qquad A_h \subseteq \mathscr{V}. \tag{10}$$

Here A_h is the allowed token set after filtering. If no filter is configured, $A_h = \mathscr{V}$. The inverse selector $\mathrm{token}_h^F(r)$ denotes the token at rank r in the same ordered allowed set. Sender and receiver must apply the identical mask and ordering because removing one candidate shifts every lower choice, a trial is unavailable if the allowed set cannot supply a requested rank.

Algorithm 1 gives the non-segmented bounded-codec RankCloak procedure. The algorithm first encodes the displayed artifact into ranks. It then generates one cover token per rank under the cover prompt. Recovery replays the same prompt and exact saved token sequence, recomputes each token's rank, and decodes the recovered ranks back to the displayed payload. This algorithm is the basic measurement unit for the non-segmented cover-generation trials.

Algorithm 1 Non-segmented RankCloak bounded-codec trial for encoding a displayed synthetic artifact as ranks, generating one cover token per rank, and recovering by exact-copy replay.

Require: Synthetic payload x; model M; cover prompt p; payload codec C; optional deterministic token filter F; shared configuration K_{common}.
Ensure: Cover text, recovered payload $\hat{x}$, exact-recovery flag, and recorded cover metrics.

$R \leftarrow C.\text{encode}(x)$
context $\leftarrow$ tokenize(p)
cover_tokens $\leftarrow$ []
for each rank $r_i \in R$ **do**
  logits $\leftarrow M.$next_logits(context)
  if F is configured **then**
    allowed_tokens $\leftarrow F$(logits)
  else
    allowed_tokens $\leftarrow$ vocabulary(M)
  end if
  $y_i \leftarrow$ token with one-indexed rank r_i among allowed_tokens
  append y_i to cover_tokens
  append y_i to context
end for
cover_text $\leftarrow$ detokenize(cover_tokens)

context $\leftarrow$ tokenize(p)
recovered_ranks $\leftarrow$ []
for each exact saved token y_i in cover_tokens **do**
  logits $\leftarrow M.$next_logits(context)
  if F is configured **then**
    allowed_tokens $\leftarrow F$(logits)
  else
    allowed_tokens $\leftarrow$ vocabulary(M)
  end if
  append rank_of(y_i, logits, allowed_tokens) to recovered_ranks
  append y_i to context
end for
$\hat{x} \leftarrow C.\text{decode}$(recovered_ranks)
exact_recovery $\leftarrow (\hat{x} = x)$
return cover_text, $\hat{x}$, exact_recovery

Before Algorithm 1 is executed, the selected representation validates payload syntax, exact byte length, codec applicability, and the source digest. The retained record saves the requested ranks, cover-token IDs, prompt and configuration identities, codec metadata, and failure position where applicable. After decoding, exact rank and representation equality are distinguished from exact serialized-payload equality, which additionally requires the declared byte length, bytes, syntax, and SHA-256 digest to match. The same generation and replay loop also applies when the validated direct-subword representation supplies R; its payload-side reconstruction and framing checks are detailed in Supplementary Note S1.

RankCloak segmented multi-cover method

The segmented multi-cover method extends RankCloak from one cover sequence to several shorter public messages. Long sequences of forced ranks can accumulate topic drift, formatting artifacts, or low-probability continuations. Segmentation restarts the cover context for each chunk, allowing the effect of distributing the payload across several messages to be measured without claiming that those messages are human-natural. The method is also useful analytically because it separates the payload-bearing forced span from the rest of the public message.

For a full rank sequence R, segmentation splits the payload into m chunks:

$$R = R^{(1)} \| R^{(2)} \| \cdots \| R^{(m)}. \tag{11}$$

Here $\|$ denotes sequence concatenation, and each $R^{(j)}$ is an ordered, non-overlapping rank chunk. The completed design includes frozen segmented schedules and an ablation over 4-, 8-, 16-, and 32-rank chunks rather than treating one chunk size as universally preferred.

For segment j, the forced span emits one token per payload rank. Recovery uses only those forced tokens:

$$y_k^{(j)} = \text{token}_{h_k^{(j)}}\left(r_k^{(j)}\right), \qquad \hat{r}_k^{(j)} = \rho_{h_k^{(j)}}\left(y_k^{(j)}\right). \tag{12}$$

When a filter is configured, both operations use the filtered ordering defined after Eq. (10). After the forced span $y^{(j)}$, the sender appends a greedy non-payload tail $z^{(j)}$. The public message is the tokenizer rendering of the concatenated token sequence $y^{(j)} \| z^{(j)}$. The receiver ignores $z^{(j)}$ and decodes only $y^{(j)}$:

$$m_j = \text{text}\left(y^{(j)} \| z^{(j)}\right), \qquad \hat{R} = \hat{R}^{(1)} \| \cdots \| \hat{R}^{(m)}. \tag{13}$$

Here $\text{text}(\cdot)$ means ordinary tokenizer rendering of the token sequence; it is not an additional cryptographic operation. The canonical expression in Eq. (13) has no lead-in. For the evaluated lead-in extension, a non-payload prefix is generated before $y^{(j)}$, and the record stores exact lead-in, forced, tail, and full token-ID arrays, a token-role mask, and half-open forced-span offsets. The decoder uses those declared offsets and does not infer a payload boundary from visible prose.

A core component of this method is the forced-span and tail separation. The payload is carried only by the forced span, while the tail is a greedy continuation whose effect on the complete public message is measured separately. This distinction prevents a misleading quality result: full-message diagnostics can change because of the non-payload tail even if the payload-bearing forced span remains low probability or exhibits surface artifacts. For that reason, the Results report forced-span metrics and full-message metrics separately.

Algorithm 2 describes the segmented multi-cover protocol. It encodes the hex-like payload, splits the rank sequence into short chunks, generates one forced span per chunk, appends the selected tail-policy output after each forced span, and recovers the payload by replaying only the forced spans. Each segment is its own replay component: generation starts from its declared prompt, and recovery reads the exact saved token IDs within its declared forced-span boundary.

Algorithm 2 Segmented multi-cover RankCloak trial for distributing a hex-nibble payload across payload-bearing forced spans with non-payload natural tails.

Require: Synthetic hex-like payload x; model M; hex-nibble codec C_{hex}; segment size s; prompt schedule $P = (p_1, \dots, p_m)$; deterministic token filter F; tail policy T; shared configuration K_{common}.
Ensure: Public messages $(\text{message}_1, \dots, \text{message}_m)$, recovered payload $\hat{x}$, exact-recovery flag, forced-span metrics, and full-message metrics.

$R \leftarrow C_{\text{hex}}.\text{encode}(x)$
split R into chunks $(R_1, \dots, R_m)$, each with at most s ranks
for each segment $j = 1, \dots, m$ **do**
  context $\leftarrow$ tokenize$(P[j])$
  forced_tokens$_j \leftarrow []$
  for each rank $r \in R_j$ **do**
    logits $\leftarrow M.$next_logits(context)
    allowed_tokens $\leftarrow F$(logits)
    $y \leftarrow$ token with one-indexed rank r among allowed_tokens
    append y to forced_tokens$_j$
    append y to context
  end for
  tail_tokens$_j \leftarrow$ TAILPOLICY$(M, \text{context}, F, T)$
  message$_j \leftarrow$ detokenize(concat(forced_tokens$_j$, tail_tokens$_j$))
end for
emit public messages $(\text{message}_1, \dots, \text{message}_m)$

recovered_chunks $\leftarrow []$
for each segment $j = 1, \dots, m$ **do**
  context $\leftarrow$ tokenize$(P[j])$
  read the exact saved tokens within the declared forced-span boundary for R_j
  ignore all tail tokens
  segment_ranks $\leftarrow []$
  for each forced token y in the payload-bearing span **do**
    logits $\leftarrow M.$next_logits(context)
    allowed_tokens $\leftarrow F$(logits)
    append rank_of(y, logits, allowed_tokens) to segment_ranks
    append y to context
  end for
  append segment_ranks to recovered_chunks
end for
$\hat{x} \leftarrow C_{\text{hex}}.$decode(concat(recovered_chunks))
exact_recovery $\leftarrow (\hat{x} = x)$
return $(\text{message}_1, \dots, \text{message}_m), \hat{x}$, exact_recovery

The non-lead-in form in Algorithm 2 supplies the canonical segment layout. The lead-in extension inserts a rank-1 prefix before the forced span; its exact token IDs must be replayed before forced-rank recovery, and its saved length shifts the half-open forced boundary. Lead-in lengths 2, 4, 8, 16, and 32 were compared with the no-lead-in condition. Single-topic segmentation reuses its assigned prompt, whereas multi-topic segmentation rotates deterministically through the declared prompt categories. Segment order, prompts, boundaries, and token roles are retained metadata rather than information inferred by the decoder.

Algorithm 3 implements the fixed natural-tail policy available to Algorithm 2. The tail is generated greedily, begins only after the payload-bearing forced span, and is not decoded. Its purpose is to measure how a non-payload continuation changes the complete public message while preserving a separate measurement of the forced span. This tail extension is a sentence-completion step rather than part of the hidden payload. A forced span can end abruptly because its tokens were selected to carry payload ranks, not necessarily to finish a phrase or sentence. The minimum length and sentence-boundary rule allow a continuation before the hard cap prevents non-payload text from growing without bound; no claim of human-perceived completeness follows from this proposed heuristic.

Algorithm 3 Greedy natural-tail policy used after each segmented forced span; the generated tail carries no payload ranks and is ignored by the decoder.

Require: Model M; current context after the forced span; deterministic token filter F.
Ensure: Non-payload tail tokens.

```
tail_tokens ← []
while |tail_tokens| < 60 do
    choose the rank-1 token z under M.next_logits(context) among tokens allowed by F
    append z to tail_tokens
    append z to context
    if |tail_tokens| ≥ 20 and detokenize(tail_tokens) ends at a sentence boundary then
        break
    end if
end while
return tail_tokens
```

Algorithm 3 realizes TAILPOLICY as a fixed sentence completion of 20 to 60 tokens. The completed primary matrix uses a separately frozen dynamic condition as its reference level: after at least eight greedy tokens, stopping requires terminal punctuation or a double newline, balanced delimiters, and no declared dangling connective or punctuation, with a 256-token emergency cap recorded as censoring. A no-tail condition returns an empty tail and all three policies are non-payload conditions ignored during decoding (their complete stopping definitions appear in Supplementary Note S1).

Evaluation corpus, controls, and metrics

The experiments use deterministic synthetic payloads that resemble cryptographic or operational artifacts while avoiding real secret material. The corpus contains 480 public deterministic test vectors, 60 in each of eight algorithm-defined classes; SHA-256 hexadecimal digests, HMAC-SHA256 hexadecimal tags, deterministic 96-bit nonces, 128-bit hexadecimal tokens, UUIDv4 identifiers, AES-256-GCM base64 outputs, ChaCha20-Poly1305 base64 outputs, and Ed25519 base64 signatures[34–36, 40–43]. Authenticated-encryption and signature rows are genuine outputs of the declared algorithms under reproducible public test inputs. No real credentials, API keys, private keys, account identifiers, operational secrets, commands, private user data, or deployed traffic are used.

Each payload is evaluated first as a representation problem and then, where applicable, as a cover-generation problem. The representation diagnostics measure how the exact serialization behaves under direct subword tokenization and how many ranks are required by the bounded codecs. Cover-generation conditions include direct subword ranks, ASCII-byte fixed-radix coding with $B = 8$, ASCII-byte fixed-radix coding with $B = 16$, raw hex-nibble coding for eligible lowercase hexadecimal payloads, and single- and multi-topic segmented hex-nibble protocols. All generation, recovery, and diagnostic rows use the deterministic rank-ordering rule in Eqs. (2) and (3) as part of the shared replay configuration.

Generation used Meta-Llama-3-8B-Instruct, Qwen2.5-7B-Instruct, and Mistral-7B-Instruct-v0.3 in pinned Q4_K_M configurations with their embedded GGUF tokenizers. Eighteen English templates span six prompt categories. Secondary Spanish and Simplified Chinese conditions evaluate saved-token recovery but do not establish multilingual human naturalness. The primary matrix contains 14,400 generated units; 6,480 RankCloak payload trials and 7,920 prompt, model, representation, and length-matched ordinary controls. The complete ledger reconciles 38,504 declared work units across the primary, robustness, ablation, multilingual, evaluator, and detector stages. Complete corpus construction, model and tokenizer identities and hashes, prompt schedules, and work-unit accounting appear in Supplementary Note S2.

The protocol variants define how encoded ranks become public text. Non-segmented trials use Algorithm 1 and replay the complete saved cover-token sequence. Segmented trials use Algorithm 2; Algorithm 3 or another declared tail condition follows each forced span, while recovery uses only the saved forced tokens. Prompt-matched controls provide comparison text for automated diagnostics and detector training; they are not human-written naturalness baselines. The deterministic safe-text filter rejects declared control, replacement, code, markup, URL, escape, and formatting fragments. The ablation compares it with no filter and an isolated roundtrip-stable filter, lead-in lengths 0, 2, 4, 8, 16, and 32, chunk sizes 4, 8, 16, and 32, dynamic, fixed, and no-tail policies, and frozen single- versus multi-topic schedules.

Matched ordinary-model controls remain the primary detector comparison because they isolate the encoding intervention under the same generator. A separate secondary corpus contains 250 human-authored responses selected deterministically from Databricks Dolly 15k and length-matched within the available prompt-template strata. These texts were used only as frozen test negatives, never for detector fitting or threshold selection. This bounded corpus comparison is not a participant study, human evaluation, or evidence of naturalness.

The Results report exact serialized-payload recovery, representation and rank diagnostics, cover length, requested-rank pressure, token log probability, deterministic surface artifacts, and automated readability diagnostics. A balanced 504-stimulus subset covers seven generation conditions and all 18 English templates; its surface measures include Flesch Reading Ease[44], frozen artifact flags, and TF-IDF prompt similarity, with intervals bootstrapped over prompt templates. Source-model and held-out-evaluator likelihoods are reported as model diagnostics, not participant ratings. No human study was performed.

For segmented rows, every cover-quality measurement is assigned to one of two scopes. Forced-span metrics are computed only over payload-bearing tokens decoded by the receiver. Full-message metrics are computed over the complete rendered message, including non-payload tails and any experimental lead-in. This scope distinction is necessary because tails can change full-message proxies while carrying no payload ranks. Declared unavailable combinations are excluded from the applicable estimand and reported separately rather than classified as observed failures. Complete metric definitions, transformation coverage, ablation schedules, and exclusion rules appear in Supplementary Notes S3 through S6.

For paired segmented trials, tail gain is the full-message mean token log probability minus the forced-span mean token log probability, $G_{\text{tail}} = \bar{\ell}_{\text{full}} - \bar{\ell}_{\text{forced}}$. Positive values mean that averaging in the non-payload continuation raises the full-message model score; they do not imply a change in the payload-bearing span. The empirical frontier reports forced-span and full-message views separately on the four-class common hexadecimal support, with payload-bootstrap uncertainty and automated surface-flag burden detailed in Supplementary Note S5.

Recovery, robustness, and capacity endpoints

The strongest recovery endpoint passes the original saved token IDs to the decoder under K_{common}. Saved IDs are an exact-copy record, not an ordinary text channel. Visible-text replay first detokenizes and retokenizes the rendered string, so an apparently unchanged string can yield different IDs or boundaries[39]. Arbitrary editing, paraphrasing, formatting changes, truncation of payload-bearing tokens, cross-model decoding, and prompt or filter search are not supported recovery modes. Controlled secondary tests separately evaluate unmodified visible text, final-tail truncation, Unicode NFKC, quote conversion, whitespace and line-ending changes, deterministic character and token edits, markdown copy/paste, paraphrase, limited canonicalization, and cross-model decoding. Transformations are frozen as separate conditions rather than composed into an uncontrolled channel. Final-tail truncation removes only declared non-payload tokens and is not evidence of arbitrary truncation tolerance. First-divergence labels localize the first recorded mismatch but do not establish its cause.

For a bounded representation containing H source bits and a rank alphabet of size B, the minimum fixed-radix forced length and nominal forced-position rate are

$$n_B = \left\lceil \frac{H}{\log_2 B} \right\rceil, \qquad R_B = \frac{H}{n_B} \le \log_2 B. \tag{14}$$

Here H is the information actually presented to that codec, eight bits per exact serialized byte for the byte codecs and four bits per canonical hexadecimal character for the nibble codec. For the byte bitstream, $n_B = \lceil 8m/\log_2 B \rceil$, consistent with Eq. (9). Direct subword ranks do not have a fixed B and are excluded from this bounded-capacity identity.

For representation comparisons on a common artifact support, we additionally report effective artifact bits per forced token, $R_{\text{art}} = H_{\text{art}}/n_{\text{forced}}$. Here H_{art} is held fixed across representations, four bits per canonical hexadecimal character in the common-hex analysis, rather than set to the codec's serialized input width. This descriptive rate permits direct and bounded representations to be compared on the same artifacts; it is not a cryptographic-security capacity.

If segment j has ℓ_j lead-in tokens and t_j tail tokens, and $\delta \ge 0$ denotes any other counted non-payload extension under the retained accounting convention, full cover length and effective rate are,

$$N_{\text{full}} = n_{\text{forced}} + \sum_{j=1}^{J} (\ell_j + t_j) + \delta, \qquad R_{\text{eff}} = \frac{H}{N_{\text{full}}} \le \frac{H}{n_{\text{forced}}} = R_B \quad (n_{\text{forced}} = n_B). \tag{15}$$

Segmentation changes context resets and auxiliary text, not the number of forced codec positions. Positive cover-length residuals are overhead rather than payload capacity.

For eligible-token probabilities $p_{(1)}(h) \ge \cdots \ge p_{(B)}(h)$, any bounded selection $r \le B$ obeys the same-context conditional quality relation

$$-\log p_{(1)}(h) \le -\log p_{(r)}(h) \le -\log p_{(B)}(h), \qquad Q_B = \frac{1}{n_B} \sum_{i=1}^{n_B} -\log p_M(y_i \mid h_i). \tag{16}$$

Some tables report mean log probability $\bar{\ell} = -Q_B$, for which larger values are less negative. These bounds concern model likelihood under identical filtered histories; they are not statements about grammar, meaning, human naturalness, security, or detectability.

The confirmatory endpoint is exact equality between the decoded serialized payload bytes and the declared source, including length, syntax, and SHA-256 digest. Exact rank and representation equality are retained as diagnostics. Trial-level Wilson 95% intervals are reported[45]; a strict payload group passes only if every constituent prompt, model, and eligible protocol trial succeeds, and a group defined by model and payload applies the same rule within one model. Segments are never counted as independent payload trials. Grouped bootstrap procedures use payload or prompt template as the resampling unit according to the estimand. Declared unavailable combinations never enter a failure denominator.

Neural steganalysis, statistical analysis, and computational overhead

Strict corpus preparation preceded feature extraction and splitting. Visible text was Unicode NFKC normalized, case-folded, Unicode whitespace collapsed, and hashed. Both members of any matched pair implicated in normalized exact duplication were removed. Near duplicates were predeclared at cosine similarity at least 0.95 using L2-normalized character-boundary TF-IDF 3–5-grams; payload groups connected by an edge formed immutable components. Seed-20260831 component assignment retained 15,838 rows as 9,904 training, 2,952 validation, and 2,982 test observations. The audit found zero cross-partition payload, matched-pair, normalized-text, component, or declared near-duplicate links. This protects the stated split under the declared rule but cannot rule out semantic paraphrases below the threshold.

The data-adaptive text-only attackers retain the fixed TextCNN-equivalent[26,46] and pinned DeBERTa-v3-base[47] architectures and hyperparameters; neither was tuned on the V3 test data. A separate model-aware logistic attacker uses saved exact-generation-model token log probabilities summarized as surprisal features, with regularization chosen on validation ROC-AUC; RankCloak-only rank fields were excluded. Matched evaluation uses all three families, whereas each leave-one-family-out fit excludes the target family from both training and validation. These adaptive and exact-model-aware attackers are strong bounded evaluations, not exhaustive threat models.

ROC-AUC uses tie-aware empirical ranks. Normalized partial AUC from 0 to 1% FPR uses the empirical step envelope without interpolating ROC vertices. Low-FPR thresholds maximize validation TPR subject to at most $\lfloor \alpha n_{\mathrm{neg}} \rfloor$ validation false positives and are then frozen for test. Reporting requires at least $1/\alpha$ validation and test negatives: 100 for 1% FPR and 1,000 for 0.1% FPR. Consequently, matched evaluation supports both targets, but family holdouts, the human-authored comparison, and the entropy and quantization subgroups do not support an empirical 0.1% result. Confidence intervals use 2,000 resamples of complete deduplication clusters[48].

The entropy-gated extension calibrates model-specific median and 75th-percentile Shannon-entropy thresholds on ordinary generation traces, without detector outcomes. At each span position, an eligible context forces the next payload rank; an ineligible context emits an ordinary top-p sample and consumes no rank. The decoder recomputes eligibility before every observed token, ignores sampled-skip ranks, appends all tokens to context, and receives no saved gate-position metadata. The ungated, median-gate, and strict 75th-percentile conditions each contain 120 RankCloak trials and 120 paired length-matched controls. Payload completion, fixed-budget payload fraction, length, saved-ID and visible-text recovery, and detection are separate endpoints; fixed-payload bits per generated token is defined only for completed cases.

The matched quantization analysis holds one pinned Qwen2.5-7B-Instruct revision, tokenizer, prompt, payload, codec, filter, target length, backend, and seeds fixed while comparing Q4_K_M with Q8_0. It replays 1,920 historical Q4 rows and pairs them with 1,920 Q8 generations. Visible-text recovery for the 960 historical Q4 RankCloak covers is computed from existing records without generating new covers; divergent retokenizations are replayed under the exact Q4 model and saved span rule. Paired recovery differences use payload-group bootstrap intervals and an exact two-sided McNemar test. Q4-to-Q8, Q8-to-Q4, and pooled detector fits retain frozen payload partitions and validation-selected thresholds.

Statistical inference respects payload and prompt-template grouping, with segments nested within covers. Paired ablations use payload as the pairing and bootstrap unit. Mixed models include payload and prompt grouping where supported, negative-binomial models for overdispersed artifact counts, and Gaussian models for continuous outcomes[49,50]. Holm adjustment controls selected contrast families; Benjamini-Hochberg values accompany exploratory families. Convergence, dispersion, zero-inflation, singularity, and residual diagnostics are retained. Three of five fitted mixed models were singular, no undeclared fixed-effects fallback was substituted, and compact ablation, topic, auxiliary-detector, and near-duplicate analyses remain labeled exploratory or post-outcome where applicable. Complete model formulas, bootstrap seeds, split definitions, diagnostics, and multiplicity details are in Supplementary Notes S5 through S7.

Generation overhead uses inclusive wrapper wall time retained by the harness, including the reported setup, generation, and supported replay scopes. Payload throughput is representation bits divided by generation time, and effective rate follows Eq. (15). Detector benchmarks report fixed-device CUDA wall/training time and peak allocated VRAM. CPU time, repeated warm-up measurements, and cross-device hardware generalization were unavailable and were not inferred. Complete timing cells and component scopes appear in Supplementary Note S9.

Results

Expanded evaluation and exact-copy recovery

The completed primary design produced 14,400 work units across three model families and 18 English prompt templates. Of these 6,480 were payload bearing RankCloak trials spanning the six protocol/codec configurations and 7,920 were ordinary controls. All 6,480 decoded serialized payloads exactly under saved-token replay (1.000000; Wilson 95% CI $[0.999408, 1.000000]$; Table 1). The stricter aggregation also passed for all 480 payload groups (CI $[0.992061, 1.000000]$) and all 1,440 groups defined by model and payload (CI $[0.997339, 1.000000]$). These grouped endpoints prevent repeated prompts and protocols from inflating the apparent number of independently generated artifacts.

The recovery question was whether codec inversion remained exact after forcing the ranks into cover contexts, rather than whether emitted ranks could merely be read back from the same loop. Success was defined at the original serialization and digest, and every model and eligible protocol stratum reached that endpoint. The all-success result therefore supports deterministic implementation fidelity across the evaluated prompt/model matrix. Its uncertainty is one-sided in substance; the data bound the rate of error under this completed design but cannot show that an unobserved replay environment or transmission channel would also be error-free.

Secondary exact-copy tests recovered 288/288 Spanish trials and 288/288 Simplified Chinese trials, or 576/576 in total. They establish recovery for those retained prompts under the same saved-token contract; they do not establish multilingual naturalness or robustness to ordinary text transfer. Across primary, ablation, robustness, multilingual, evaluator, and detector work, the final ledger reconciled 38,504/38,504 units: 38,072 succeeded and 432 were declared unavailable. No unit remained and none was classified as an execution failure. Recovery strata and secondary-language endpoints appear in Supplementary Tables S5 and S6.

Design or endpoint	Scope	Result
Payload corpus	Eight algorithm-defined classes, 60 public deterministic artifacts per class	480 artifacts
Model and prompt coverage	Three open-source 7B to 8B families; 18 English templates in six categories	14,400 primary work units (6,480 payload; 7,920 control)
Primary exact-copy recovery	Serialized payload, saved-token replay	6,480/6,480; 1.000000 (95% CI $[0.999408, 1.000000]$)
Strict grouped sensitivity	Payload groups; groups defined by model and payload	480/480 (CI $[0.992061, 1.000000]$); 1,440/1,440 (CI $[0.997339, 1.000000]$)
Spanish secondary recovery	Saved-token replay	288/288 (CI $[0.986837, 1.000000]$)
Simplified Chinese secondary recovery	Saved-token replay	288/288 (CI $[0.986837, 1.000000]$)
Complete work ledger	All declared revision matrices	38,504/38,504 reconciled: 38,072 success; 432 unavailable; 0 execution failure

Table 1. Evaluation design and recovery endpoints. Confidence intervals are trial- or group-level Wilson 95% intervals as identified. Recovery requires saved-token exact-copy replay with the declared shared configuration. Unavailable work units are configuration/retained-output absences, not observed decoding failures.

Payload representation, rank pressure, and capacity tradeoff

Figure 1 compares payload representations across 480 unique artifacts in eight classes and three models. The 1,440 direct observations across payloads and models contained 60,861 direct-rank positions. Direct-subword representations required 11 to 77 forced symbols, with class medians ranging from 20 to 66, and the largest observed direct rank was 145,498. These counts and ranks depend on the model and tokenizer rather than defining a fixed alphabet. The deterministic codecs instead imposed rank ceilings of 8 or 16 while generally increasing forced length. On the four-class common hexadecimal support, median effective artifact rates were approximately 4.571 for direct subword, 4.000 for hex nibble, 2.000 for ASCII $B = 16$, and 1.497 for ASCII $B = 8$ artifact bits per forced token. These descriptive rates compare representations on identical eligible artifacts; they are not cryptographic-security capacities. The whiskers in Fig. 1 are observed 5th to 95th percentiles, not confidence intervals. The detailed theoretical capacity and tail validation tests a different question: whether retained records satisfy the declared algebraic identities and conditional bounds (in Supplementary Fig. S6 and Table S14). The common-support empirical capacity and quality frontier follows in Fig. 2.

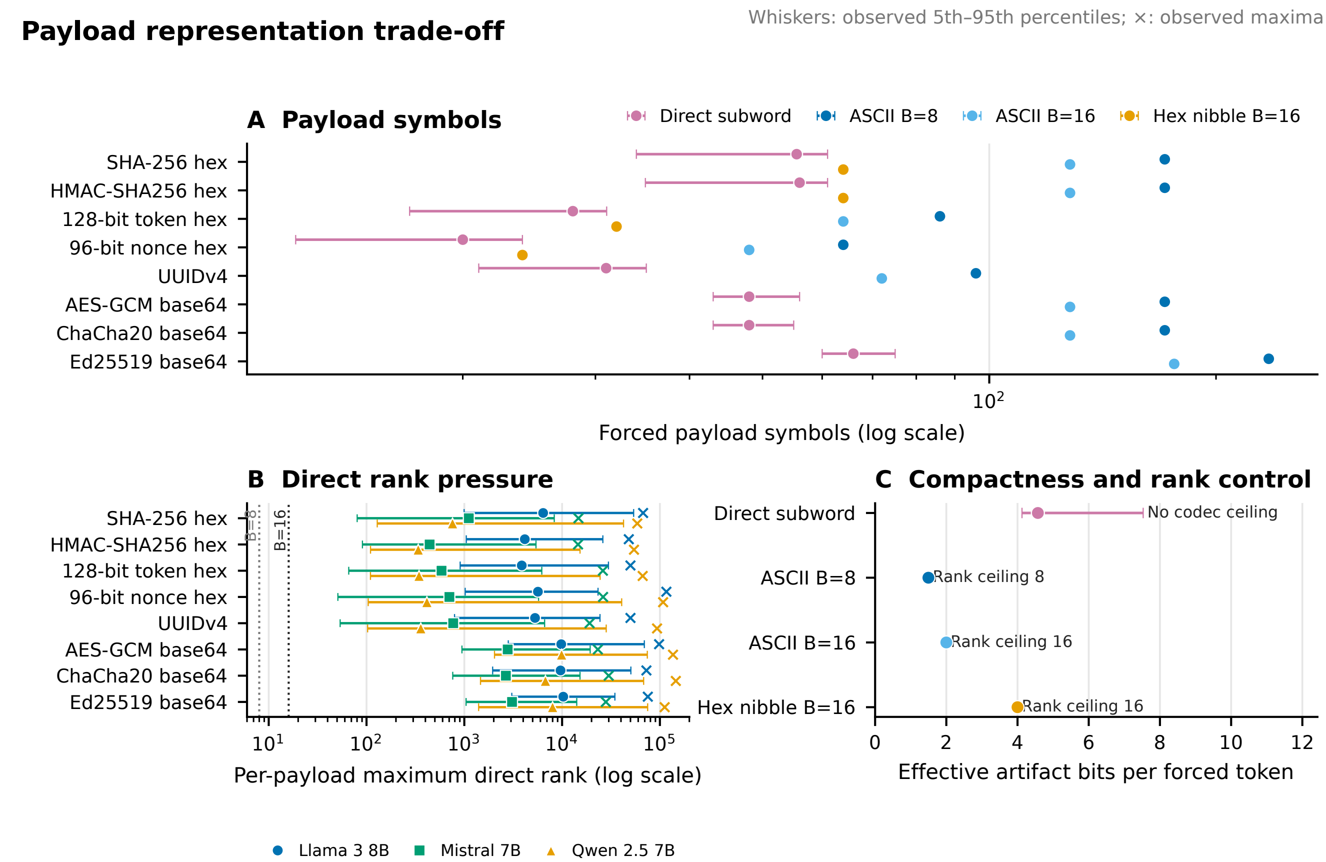

Figure 1. Payload representation, rank pressure, and capacity tradeoff. Direct-subword symbol counts and rank pressure are summarized for 1,440 observations across payloads and models comprising 60,861 direct-rank positions. Their values are tokenizer and model dependent. Bounded codecs constrain requested ranks to at most 8 or 16, at the cost of more forced positions. Effective artifact-rate comparisons use the four-class common hexadecimal support and are descriptive rather than cryptographic-security capacities. Whiskers show observed 5th to 95th percentiles, not confidence intervals.

Figure 2 compares mean token count with mean source-model token log probability over 4,320 trials on the four-class common hexadecimal support. Three models and six protocols form 18 model and protocol cells, and separate forced-span and full-message views give 36 plotted cells. Compact direct-subword spans generally occupy the low-length region but can have poorer source-model likelihood, whereas bounded protocols trade longer forced spans for controlled rank alphabets. Segmented full-message points move toward better full-message likelihood by adding non-payload natural-tail length; this movement is not improved likelihood on the payload-bearing forced span. Higher mean token log probability is better; point area represents mean automated surface-flag burden, and uncertainty uses 2,000-resample payload-bootstrap 95% confidence intervals. The complete primary matrix contains no lead-in condition, so exploratory lead-in rows were not pooled. These automated diagnostics are not human-perceived judgments, and the empirical frontier is distinct from the theoretical capacity and tail validation.

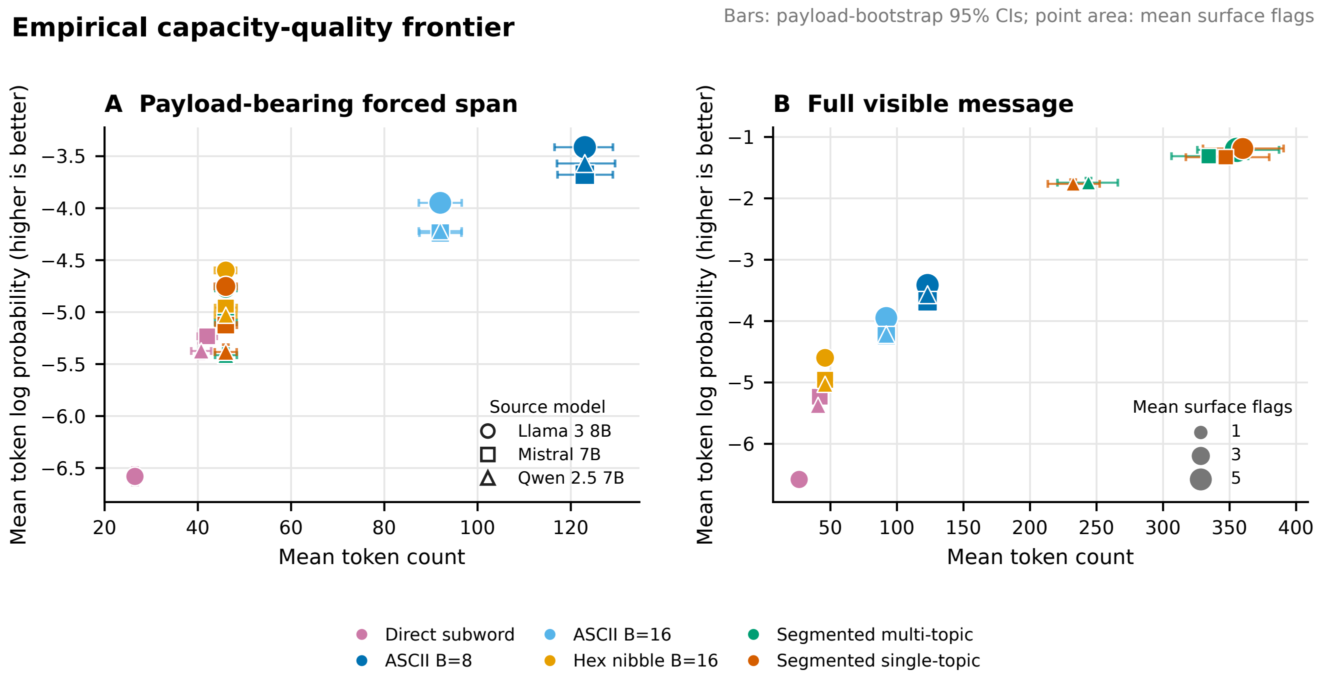

Figure 2. Empirical capacity and quality frontier. Mean token count is shown against mean source-model token log probability for 4,320 trials on four-class common hexadecimal support, spanning three models, six protocols, 18 model and protocol cells, and 36 forced-span or full-message cells. Higher mean token log probability is better; point area represents mean automated surface-flag burden. Error bars are 2,000-resample payload-bootstrap 95% confidence intervals. The primary matrix contains no lead-in condition. Likelihood and surface flags are automated diagnostics, not human judgments, and this empirical frontier is distinct from the theoretical capacity and tail validation in Supplementary Fig. S6.

Exact-copy replay and transmission fragility

Replay mode materially changed recovery (Fig. 3). Saved-token replay recovered 144/144 robustness covers, whereas detokenizing the same covers and retokenizing their visible text recovered 88/144 (0.611111). So even an unedited visible string did not guarantee the original token sequence. Raw unmodified transmission recovered 144/144, but common transformations were damaging. Unicode NFKC recovered 82/144 (0.569444), quote conversion 56/144 (0.388889), and whitespace trimming 28/144 (0.194444). Markdown copy/paste and paraphrase recovered 0/144, and the two cross-model decoder outcomes for each cover recovered 0/288.

These conditions address a different question from the primary result of whether a cover can leave its saved-token record and still recreate the forced ranks. The answer depended sharply on the frozen channel definition. The saved-ID, visible-retokenization, and raw-transmission rows are distinct replay procedures and should not be pooled. Their contrast shows why the receiver contract must name the tokenization path, not merely say that the visible text was copied exactly. Cross-model zero recovery further shows that similar prompt intent does not substitute for model and tokenizer identity.

Deleting only the declared final 10% of a message recovered 144/144 because that operation was constrained to non-payload tail tokens. It is not evidence of arbitrary truncation tolerance; removing a forced token or earlier context would change the decoded sequence. The broader matrix also showed low recovery after token or character edits and line-ending conversion (Supplementary Note S4). Limited canonicalization preserved unmodified controls but did not recover the available transformed cells, so it did not demonstrate a general mitigation.

Tail-only truncation is consequently a boundary check on the decoder definition rather than a successful error-correction result. The decoder is designed to ignore tails, so removing only tail labels should be harmless. By contrast, prefix changes and edits inside a forced span can alter a segment's initial tokenization or model history and propagate. The failed limited-canonicalization cells show that normalizing a small declared set after the fact was insufficient in this matrix; no broader mitigation is inferred.

Relative to the reviewer-requested inventory, five transformation classes were tested directly, three had only partial proxies, and three were untested. In particular, arbitrary prefix/suffix insertion, general case conversion, and general punctuation edits were not tested; line wrapping, email/markdown quoting, and sentence-boundary changes were only partially represented. First-divergence labels locate the first observed mismatch among tokenization, context, rank, or payload stages, but are descriptive and cannot by themselves establish why a transformation failed (Supplementary Fig. S1 and Supplementary Tables S7 and S8).

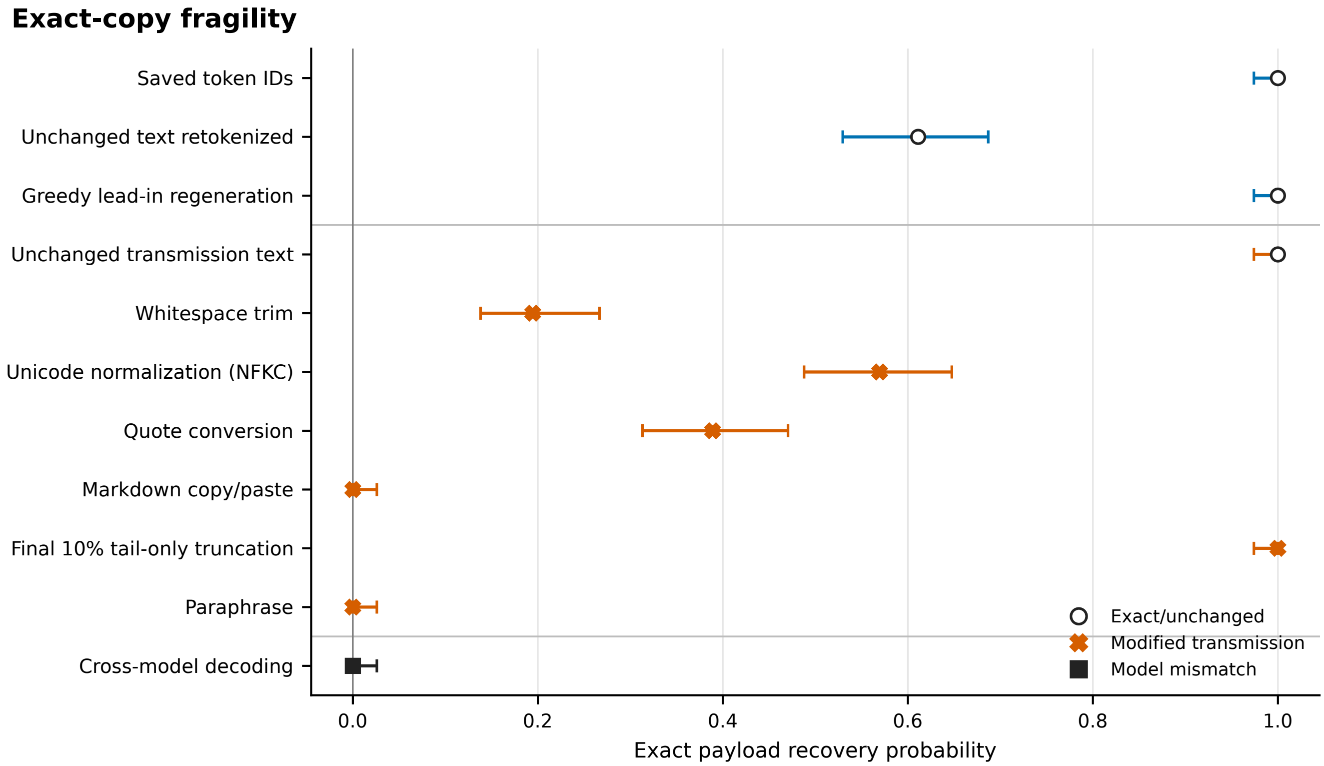

Figure 3. Exact-copy fragility. Recovery proportions and Wilson 95% intervals are shown for the principal replay and transmission conditions. Saved token IDs preserve the strongest replay contract. Visible-text detokenization and retokenization already reduce recovery. NFKC normalization, quote conversion, and trimming reduce it further, while markdown copy/paste, paraphrase, and cross-model decoding yield no successful payloads. Final 10% tail truncation removes only declared non-payload tail tokens and must not be interpreted as arbitrary truncation tolerance. The full transformation matrix, denominators, limited canonicalization cells, and first-divergence diagnostics appear in Supplementary Figure S1 and Supplementary Note S4.

Forced span versus full message

Across 1,440 paired segmented trials, overall forced-span mean token log probability was -5.090904 and overall full-message mean token log probability was -1.424774, for an overall paired gain of +3.666130 (Fig. 4). Each of the six model and topic schedule cells contained 240 paired trials, and all six 2,000-resample paired payload-bootstrap confidence intervals for the gain excluded zero.

Natural tails contain no payload ranks and add no payload capacity. The higher full-message average therefore reflects the inclusion of non-payload greedy continuations and must not be interpreted as improved quality on the payload-bearing forced span. Automated readability and surface diagnostics remain in Supplementary Fig. S2 and Note S5; they are not human evaluation.

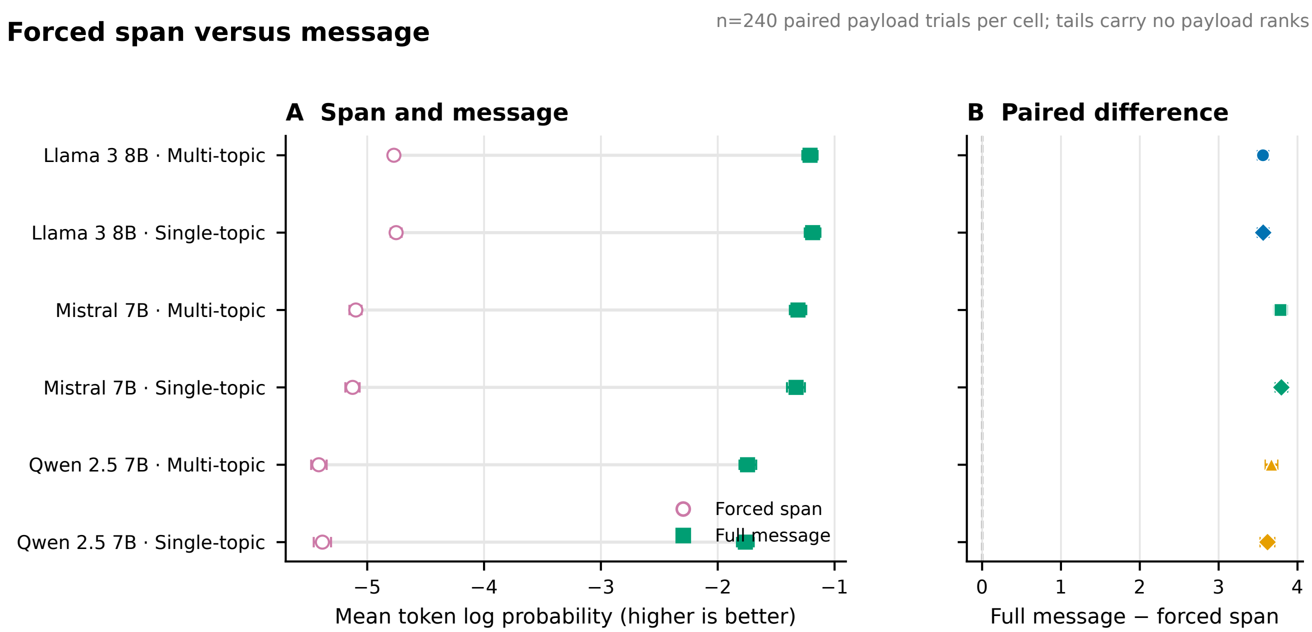

Figure 4. Forced span versus full message. Mean token log probability is paired within 1,440 segmented trials across three models and single-topic or multi-topic schedules, with 240 trials per model schedule cell. Error bars are 2,000-resample paired payload-bootstrap 95% confidence intervals; all six intervals for the full-minus-forced gain exclude zero. Natural tails contain no payload ranks and add no payload capacity, so the improved full-message average is not an improvement to the payload-bearing forced span.

Ablations, dynamic stopping, and prompt-topic effects

The locked compact ablation extraction compared 48 paired payload groups across three models and is exploratory/post-outcome (Table 2). A 32-token lead-in, relative to no lead-in, reduced mean log probability by 1.652327 (95% CI $[-1.749091, -1.553103]$) and added 159.590278 full-message tokens (CI $[125.829861, 191.030035]$). This adverse result is consistent with added context and boundary cost; the contrast does not prove a single failure mechanism. Increasing the segment size from 8 to 32 ranks increased effective rate by 1.231461 bits per full token (CI $[1.073116, 1.388188]$) and reduced full length by 174.500000 tokens (CI $[-208.405035, -140.573264]$), trading fewer tails for longer forced spans. The lead-in and segment results locate different costs. A lead-in precedes the forced span, consumes visible tokens, and changes the distribution used for every encoded rank; it therefore cannot be treated as a harmless stylistic prefix. Larger segments keep the forced count fixed but require fewer prompt resets and tail opportunities. Their higher effective rate is consistent with reduced overhead, while the forced span itself lasts longer before an ordinary greedy continuation begins.

Removing the safe-text filter changed mean log probability by approximately -0.0018 (canonical CSV estimate -0.001797; CI $[-0.037738, 0.035394]$) and effective rate by -0.062969 bits per full token (CI $[-0.134265, 0.007651]$); both intervals included zero. The Mistral roundtrip-stable-filter cell was unavailable for 48 work units, although the no-filter contrast included all three models. The filter results therefore do not establish a quality benefit. The near-zero selected filter estimates are important null evidence. The deterministic filter remains necessary to define an agreed candidate set for the canonical protocol, but this ablation did not show that it improved the selected likelihood or rate endpoints. Nor can the unavailable stable-filter Mistral condition be treated as evidence for or against that condition and it is an absence caused by the frozen isolated-vocabulary criterion. Compared with dynamic stopping, no tail increased effective rate by 3.171551 bits per full token and shortened messages by 254.951389 tokens. This arithmetic consequence of removing non-payload text is not evidence of better cover quality. A fixed sentence-tail policy of 20 to 60 tokens shortened messages by 65.708333 tokens relative to dynamic stopping, whereas its selected effective-rate interval included zero. Dynamic stopping remains a declared heuristic with an emergency cap, not a semantic judgment by readers.

The tail ablation also prevents a misleading optimization conclusion. Maximizing bits per full token alone selects no tail because every added non-payload token enlarges the denominator. RankCloak introduced tails for a different, unverified purpose: allowing a forced fragment to continue to a rule-defined stopping point. Without human ratings, the experiment measures the length/rate price of that choice but cannot determine whether the price purchases perceived naturalness. Across

15 locked topic contrasts, one Llama artifact-count comparison excluded the null; multi-topic versus single-topic expected count ratio 1.276361 (95% CI $[1.114743, 1.461409]$; Holm-adjusted $p = 0.003294$). The other 14 adjusted p-values were at least 0.396248. This compact extraction was performed post-outcome without refitting models and is exploratory as it does not support a general prompt-topic advantage. The solitary non-null row concerns one model and one artifact-count outcome. It did not recur across the other model/outcome combinations after adjustment. Accordingly, prompt diversity is retained as design coverage rather than promoted as a performance-enhancing intervention. Reviewer-relevant ablations, exploratory tail-gain subgroups, and all topic contrasts appear in Supplementary Figs. S3 and S4 and Supplementary Tables S10 and S11; the complete 60-row contrast table remains machine-readable in the public repository.

Analysis	Sample/group	Principal estimate	Uncertainty or adjusted test	Classification	Main limitation
32-token lead-in vs none	48 paired payloads; 3 models	Mean log probability -1.652327; full length +159.590278	CIs $[-1.749091, -1.553103]$; $[125.829861, 191.030035]$	Exploratory, post-outcome	Boundary/context interpretation is not causal
Segments of 32 versus 8 ranks	Same	Effective rate +1.231461 bits/full token; length -174.500000	CIs $[1.073116, 1.388188]$; $[-208.405035, -140.573264]$	Exploratory, post-outcome	Fewer tails accompany longer forced spans
No filter vs safe filter	Same	Log probability -0.001797; rate -0.062969	CIs $[-0.037738, 0.035394]$; $[-0.134265, 0.007651]$	Exploratory, post-outcome	Both selected intervals include zero; stable-filter Mistral cell unavailable
Tail policy	Same	No tail: rate +3.171551, length -254.951389; fixed tail: length -65.708333	Selected 95% CIs in Supplementary Note S6	Exploratory, post-outcome	Rate gain from deleting non-payload text is not quality gain
Topic schedule	15 locked contrasts	Llama artifact-count ratio 1.276361	CI $[1.114743, 1.461409]$; Holm $p = 0.003294$; other $p_{\mathrm{Holm}} \geq 0.396248$	Exploratory, post-outcome	One selected model/outcome contrast excludes null
Generation overhead	18 model and protocol cells; 12,144 matched rows	Mean $[1.674368, 16.087628]$ s; $[13.631855, 100.851772]$ bit/s	Payload-bootstrap intervals in Supplementary Note S9	Supporting measurement	Inclusive wall time; no CPU time or warm-up repeats

Table 2. Compact reviewer-requested analysis and operational summary. Differences are level minus canonical unless otherwise stated. Ablation and topic rows are a locked exploratory/post-outcome extraction from retained analyses; no new model was fit. Reviewer-relevant ablations, complete topic rows, availability, and timing cells are in the Supplementary Information; the full 60-row contrast table remains machine-readable in the repository.

Neural steganalysis

On the rebuilt strictly deduplicated matched test, TextCNN ROC-AUC was 0.9868 (95% CI $[0.9819, 0.9913]$), DeBERTa-v3-base was 0.9994 (CI $[0.9984, 1.0000]$), and the exact-model-aware surprisal attacker was 0.999995 (CI $[0.999970, 1.000000]$; Fig. 5). Their normalized empirical 0–1% partial AUCs were 0.6652, 0.9948, and 0.999460. High performance is adverse evidence for concealment. The model-aware result further shows that an attacker with the exact generator's likelihoods has a stronger signal than visible text alone.

Leave-one-family-out ROC-AUC ranged from 0.9186 to 0.9784 for TextCNN, 0.9972 to 1.0000 for DeBERTa, and 0.999943 to 1.000000 for the model-aware detector. These fits exclude the target family from training and validation and therefore test unseen-family transfer within the three-family corpus, not arbitrary future models. At validation-frozen nominal 1% FPR thresholds, matched test TPR/observed FPR was 0.8317/0.0127, 0.9913/0.0007, and 0.9987/0.0020, respectively. Family-holdout observed FPR reached 35/497 for TextCNN, 28/497 for DeBERTa, and 3/497 for the model-aware detector, demonstrating calibration drift. The matched test supports empirical 0.1% reporting; each family holdout has only 497 test negatives, so its 0.1% result is unavailable.

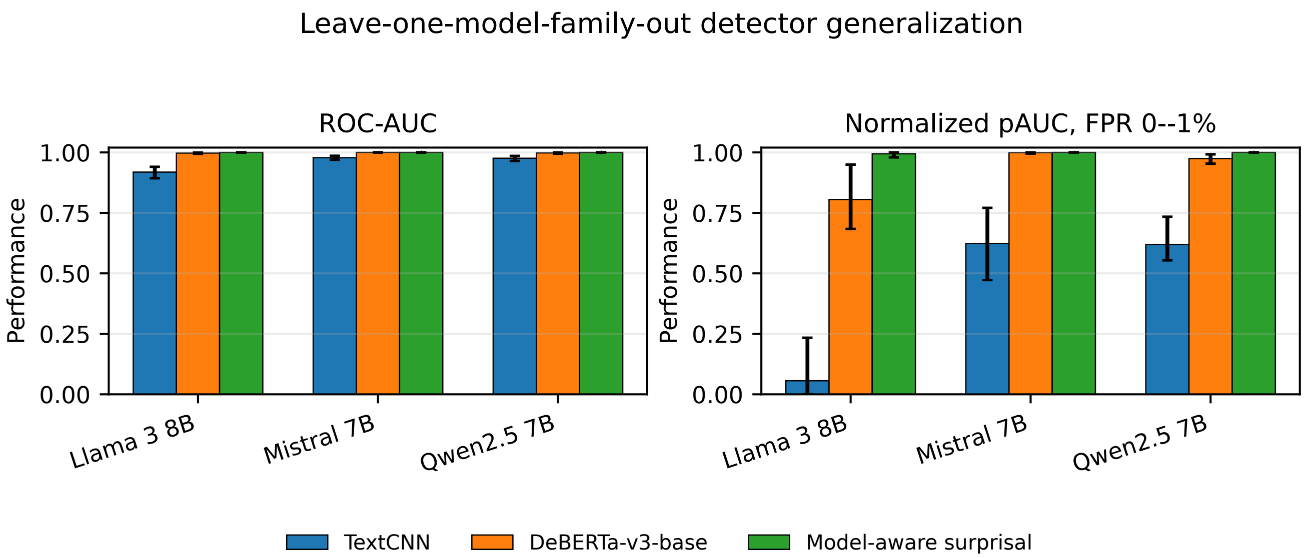

Figure 5. Strictly deduplicated detector evaluation. Matched and leave-one-family-out ROC-AUC results are shown for two data-adaptive text-only detectors and the exact-model-aware surprisal attacker. Higher performance is adverse for concealment. Components connected by declared exact or near-duplicate relations do not cross partitions; the rule does not detect semantic paraphrases below its threshold. Supplementary Note S12 reports full split counts, validation-frozen low-FPR operating points, exact false-positive counts, and unavailable 0.1% conditions.

Confirmatory controls, entropy gating, and quantization

The human-authored secondary analysis selected 250 Dolly responses across 16 available templates, with no exact or declared near-duplicate links within that corpus or to the primary corpus. Against these negatives, ROC-AUC was 0.7537 for TextCNN and 0.9846 for DeBERTa. At thresholds selected only on matched ordinary-model validation controls, 152/250 and 97/250 human texts were false positives. These results neither constitute a human study nor isolate RankCloak from generic machine-versus-human cues.

Entropy gating produced the expected bounded tradeoff rather than eliminating the adverse signal (Table 3). From ungated through median and strict gates, mean fixed-payload capacity conditional on completion declined from 7.4001 to 5.7920 and 4.0513 bits/token, while mean length ratio increased from 1.0000 to 1.3743 and 2.7615. The strict condition completed 114/120 payloads, with six maximum-budget failures; its capacity denominator is those 114 completed cases, not all 120 attempts. Strict-gate text-only ROC-AUC declined to 0.8850 and 0.8893 in this matrix, but model-aware ROC-AUC remained 0.9904. Visible-text recovery was 62/120, 65/120, and 68/120; these small differences do not establish reliable visible-text transport or an undetectable channel.

For the single pinned Qwen revision, saved-ID recovery was 960/960 for both Q4_K_M and Q8_0. Visible-text recovery was 732/960 (76.25%) and 758/960 (78.96%). The paired Q8-minus-Q4 difference was 2.71 percentage points (grouped-bootstrap 95% CI $[-0.73, 6.35]$); paired outcomes were 588 both, 144 Q4 only, 170 Q8 only, and 58 neither, with exact two-sided McNemar $p = 0.1582$. Thus the data do not support a Q8 advantage. Independent generated paths diverged substantially (mean positionwise match 0.0155), while text-only and model-aware cross-quantization detection remained very strong and frozen nominal thresholds sometimes drifted.

Analysis or condition	Recovery or completion	Capacity and length	Detection or paired inference
Human-authored controls	250 frozen test negatives; no human labels used for fitting	Length-matched within 16 available templates	TextCNN/DeBERTa AUC 0.7537/0.9846; false positives 152/250 and 97/250
Entropy ungated	Complete/saved ID 120/120; visible 62/120	7.4001 bit/token, $n = 120$, conditional on completion; length ratio 1.0000	AUC TextCNN/DeBERTa/model-aware 0.9953/0.9999/1.0000
Entropy median gate	Complete/saved ID 120/120; visible 65/120	5.7920 bit/token, $n = 120$, conditional on completion; length ratio 1.3743	AUC 0.9825/0.9785/0.9999
Entropy strict gate	Complete/saved ID 114/120; visible 68/120; six budget failures	4.0513 bit/token, $n = 114$, conditional on completion; length ratio 2.7615	AUC 0.8850/0.8893/0.9904
Q4_K_M versus Q8_0	Saved ID 960/960 each; visible 732/960 versus 758/960	Independent-path position match 0.0155	Difference 2.71 points, CI $[-0.73, 6.35]$; McNemar $p = 0.1582$
Cross-quantization detection	Q4-to-Q8 and Q8-to-Q4 held-payload tests	One Qwen revision; two quantizations	AUC TextCNN 0.9927/0.9944; DeBERTa and model-aware 1.0000/1.0000

Table 3. Compact summary of the targeted V3 analyses. Entropy capacity is fixed-payload bits per generated token and is explicitly conditional on completion. Entropy recovery proportions use all 120 attempted cases. Detector triples are TextCNN, DeBERTa-v3-base, and exact-model-aware surprisal. Empirical 0.1% operating points are unavailable for the 250-negative human comparison and for every entropy and quantization subgroup because each has fewer than 1,000 negatives. Detailed methods, denominators, confidence intervals, thresholds, and condition-level results appear in Supplementary Notes S11–S15.

Computational overhead and generalizability

Across the 18 primary model and protocol cells, mean inclusive generation time ranged from 1.674368 to 16.087628 seconds. Mean payload throughput ranged from 13.631855 to 100.851772 bits/s, and effective rate from 0.710004 to 7.322003 bits per full token. These saved wall times include wrapper setup and decoding scope defined by the harness; CPU time and repeated warm-up measurements were not retained.

The approximately order-of-magnitude spread across cells shows that representation and model choice materially affect operational cost in the retained environment. Compact direct ranks can reduce forced work, whereas byte expansion and repeated segmented continuations increase it. Throughput and effective rate answer different questions: the first normalizes payload bits by wall time, while the second normalizes by visible full-message tokens. Neither is a deployment benchmark outside the recorded wrapper and device. On the fixed CUDA benchmark, representative TextCNN wall/training times were 91.358/9.498 seconds with 306.25 MB peak allocated VRAM; DeBERTa-v3-base times were 1,536.661/1,425.063 seconds with 6.38 GB. Those device-specific measurements do not generalize across hardware. Evaluation across three model families and exact-copy Spanish and Simplified Chinese trials broadens the controlled design.

The detector timings also illustrate the cost of the stronger adverse test: the pretrained architecture required substantially more saved time and memory than TextCNN in the single retained benchmark. CPU neural training was infeasible and was not used as a comparison point. These measurements support transparent study accounting, not a claim that the approach is efficient enough for an operational sender, receiver, or monitor (Supplementary Fig. S7 and Supplementary Tables S15 and S16).

Discussion

RankCloak provides a reproducible method for converting serialized cryptographic artifacts into language-model next-token ranks and measuring the cover text that realizes them. Its representation choices expose a central trade-off: direct subwords can be compact but leave the rank pressure uncontrolled, whereas ASCII-byte and hex-nibble codecs bound the permitted rank by spending more forced positions. Segmentation redistributes those positions across contexts, and natural tails add non-payload length. The formal and empirical capacity relationships make these costs explicit rather than treating longer cover text as additional payload capacity. Complete recovery in the evaluated primary and secondary-language matrices establishes deterministic codec and replay fidelity under the saved-token exact-copy contract. It does not establish an ordinary text channel. Detokenized visible text could change tokenization without an apparent edit, and common normalization, formatting, rewriting, or model-mismatch conditions sharply reduced or eliminated recovery. The contrast between perfect saved-token replay and fragile visible-text transmission is therefore not a secondary implementation detail; it defines the boundary of the recovery claim.

Cover-quality evidence remained protocol-dependent. Full-message mean log probability was higher than forced-span mean across paired segmented trials, but the added natural tails carry no ranks and the gain does not improve the payload-bearing positions or add capacity. Automated readability, surface diagnostics, and model-based contrasts remain computational proxies rather than human judgments, and no participant evidence supports a claim of human-perceived naturalness. Supporting analyses reinforce this interpretation without identifying a single optimum. Larger segments improved efficiency by reducing repeated tail overhead, while lead-ins worsened likelihood and length in the selected contrasts; removing tails mechanically improved rate but did not test perceived quality. These exploratory comparisons show that optimization depends on which cost is prioritized rather than establishing a universally preferable protocol. Evaluation across three model families, diverse prompts, and two secondary languages broadens the controlled setting but cannot be extrapolated to arbitrary models, languages, or deployed channels.

Steganalysis supplies the clearest adverse finding. Strict component-level deduplication and genuine leave-one-family-out evaluation reduce leakage and distribution-overlap concerns under the declared audit, but both the pretrained text detector and exact-model-aware attacker remain extremely strong. Validation-frozen low-FPR results also show that ranking performance does not ensure threshold calibration after family or quantization shift. The human-authored control comparison is informative but secondary: its high false-positive rates reveal control-domain sensitivity, and it can reflect generic machine-versus-human cues rather than RankCloak alone.

Entropy gating reduces text-only detectability in the tested matrix by reserving low-entropy positions for ordinary sampling, but it simultaneously lowers capacity, lengthens output, introduces strict-gate completion failures, and leaves a strong exact-model-aware signal. The Q4/Q8 comparison likewise shows perfect replay only for saved IDs, materially weaker visible-text recovery, substantial divergence between independently generated paths, and no statistically supported Q8 recovery advantage. Both experiments refine the boundary of the framework rather than overcoming it.

Despite these adverse results, the study contributes a controlled framework for separating representation, recovery assumptions, capacity, cover length, automated quality, transformation sensitivity, and detectability. The expanded payload, prompt, model, control, attacker, entropy, and quantization evaluations show how these quantities can be measured without equating model likelihood with reader judgment or deterministic replay with communication. RankCloak therefore establishes a measurement framework and reproducible failure surfaces for rank-based encoding, but it does not establish an undetectable, robust, secure, deployable, or practical covert communication system.

Responsible use and limitations

RankCloak is dual use and can encode sensitive-looking artifacts, but it does not provide encryption, authentication, confidentiality, or a cryptographic-security proof. Saved-token replay requires token IDs and the relevant model, tokenizer, quantization, prompt rendering, filters, and span boundaries; it is materially different from reliable communication through ordinary visible text. Visible-text recovery, paraphrasing, markdown or format transfer, and cross-model decoding remain fragile or unsuccessful under the tested conditions. No participant study, human-perceived naturalness evaluation, operational deployment, broad multilingual study, or testing on larger proprietary models was performed.

Strict deduplication is limited to the declared normalization and character-n-gram threshold and cannot rule out semantic paraphrases. Human-authored controls cover only 250 Dolly responses and may expose generic machine-versus-human signals. Leave-one-family-out, data-adaptive text-only, and exact-model-aware evaluations are strong but not exhaustive attackers. Low-FPR resolution is unavailable when fewer than 1,000 negatives are present. The entropy study uses 120 cases per gate, two templates, and thresholds calibrated from 768 positions per model; strict capacity is conditional on 114 completed cases. The quantization study covers one Qwen revision, one backend, and only Q4_K_M and Q8_0. These added evaluations do not demonstrate an undetectable, robust, secure, deployable, or practical covert channel; detailed boundaries appear in Supplementary Notes S11–S15.

Data and Code availability

The RankCloak source code and computational data are available at the GitHub repository, `https://github.com/mantzaris/llm-rankcloak`. An earlier code-and-data release is archived in Zenodo at `https://doi.org/10.5281/zenodo.21987450`. This statement does not assert that the existing DOI version contains the final V3 manuscript commit. Before submission, the final accepted computational and manuscript revision will be versioned in a recognized DOI-assigning repository and the DOI wording will be synchronized with the cited code revision.

References

1. Norelli, A. & Bronstein, M. M. LLMs can hide text in other text of the same length. In *The Fourteenth International Conference on Learning Representations* (2026).
2. Simmons, G. J. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 51–67, DOI: 10.1007/978-1-4684-4730-9_5 (Springer, 1984).
3. Cachin, C. An information-theoretic model for steganography. In *Information Hiding*, 306–318, DOI: 10.1007/3-540-49380-8_21 (Springer, 1998).
4. Hopper, N. J., Langford, J. & von Ahn, L. Provably secure steganography. In *Advances in Cryptology – CRYPTO 2002*, 77–92, DOI: 10.1007/3-540-45708-9_6 (Springer, 2002).

5. Petitcolas, F. A. P., Anderson, R. J. & Kuhn, M. G. Information hiding: A survey. *Proc. IEEE* **87**, 1062–1078, DOI: 10.1109/5.771065 (1999).

6. Fang, T., Jaggi, M. & Argyraki, K. Generating steganographic text with LSTMs. In *Proceedings of ACL 2017, Student Research Workshop*, 100–106, DOI: 10.18653/v1/P17-3017 (Association for Computational Linguistics, 2017).

7. Yang, Z.-L., Guo, X.-Q., Chen, Z.-M., Huang, Y.-F. & Zhang, Y.-J. RNN-Stega: Linguistic steganography based on recurrent neural networks. *IEEE Transactions on Inf. Forensics Secur.* **14**, 1280–1295, DOI: 10.1109/TIFS.2018.2871746 (2019).

8. Ziegler, Z. M., Deng, Y. & Rush, A. M. Neural linguistic steganography. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 1210–1215, DOI: 10.18653/v1/D19-1115 (Association for Computational Linguistics, 2019).

9. Dai, F. Z. & Cai, Z. Towards near-imperceptible steganographic text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4303–4308, DOI: 10.18653/v1/P19-1422 (Association for Computational Linguistics, 2019).

10. Shen, J., Ji, H. & Han, J. Near-imperceptible neural linguistic steganography via self-adjusting arithmetic coding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 303–313, DOI: 10.18653/v1/2020.emnlp-main.22 (Association for Computational Linguistics, 2020).

11. Zhang, S., Yang, Z., Yang, J. & Huang, Y. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 3046–3055, DOI: 10.18653/v1/2021.findings-acl.268 (Association for Computational Linguistics, 2021).

12. Wang, Y., Song, R., Li, L., Zhang, R. & Liu, J. Dynamically allocated interval-based generative linguistic steganography with roulette wheel. *Appl. Soft Comput.* **176**, 113101, DOI: 10.1016/j.asoc.2025.113101 (2025).

13. Wu, J., Wu, Z., Xue, Y., Wen, J. & Peng, W. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 10345–10353, DOI: 10.1145/3664647.3680562 (Association for Computing Machinery, 2024).

14. Lin, K., Luo, Y., Zhang, Z. & Luo, P. Zero-shot generative linguistic steganography. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 5168–5182, DOI: 10.18653/v1/2024.naacl-long.289 (Association for Computational Linguistics, 2024).

15. Kirchenbauer, J. *et al.* A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 17061–17084 (PMLR, 2023).

16. Cai, S., Ding, L. & Tao, D. Entropy-guided watermarking for LLMs: A test-time framework for robust and traceable text generation. *arXiv preprint arXiv:2504.12108* DOI: 10.48550/arXiv.2504.12108 (2025).

17. Wayner, P. Mimic functions. *Cryptologia* **16**, 193–214, DOI: 10.1080/0161-119291866883 (1992).

18. Wayner, P. Strong theoretical steganography. *Cryptologia* **19**, 285–299, DOI: 10.1080/0161-119591883962 (1995).

19. Chapman, M. & Davida, G. I. Hiding the hidden: A software system for concealing ciphertext as innocuous text. In *Information and Communications Security: First International Conference, ICICS 1997*, vol. 1334 of *Lecture Notes in Computer Science*, 335–345, DOI: 10.1007/BFb0028489 (Springer, 1997).

20. Bennett, K. Linguistic steganography: Survey, analysis, and robustness concerns for hiding information in text. Tech. Rep. CERIAS TR 2004-13, Purdue University, Center for Education and Research in Information Assurance and Security (2004).

21. Chang, C.-Y. & Clark, S. Practical linguistic steganography using contextual synonym substitution and a novel vertex coding method. *Comput. Linguist.* **40**, 403–448, DOI: 10.1162/COLI_a_00176 (2014).

22. Weinberg, Z. *et al.* StegoTorus: A camouflage proxy for the Tor anonymity system. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 109–120, DOI: 10.1145/2382196.2382211 (Association for Computing Machinery, 2012).

23. Dyer, K. P., Coull, S. E., Ristenpart, T. & Shrimpton, T. Protocol misidentification made easy with format-transforming encryption. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 61–72, DOI: 10.1145/2508859.2516657 (Association for Computing Machinery, 2013).

24. Zander, S., Armitage, G. & Branch, P. A survey of covert channels and countermeasures in computer network protocols. *IEEE Commun. Surv. Tutorials* **9**, 44–57, DOI: 10.1109/COMST.2007.4317620 (2007).

25. Badar, L. T., Carminati, B. & Ferrari, E. A comprehensive survey on stegomalware detection in digital media, research challenges and future directions. *Signal Process.* **231**, 109888, DOI: 10.1016/j.sigpro.2025.109888 (2025).

26. Wen, J., Zhou, X., Zhong, P. & Xue, Y. Convolutional neural network based text steganalysis. *IEEE Signal Process. Lett.* **26**, 460–464, DOI: 10.1109/LSP.2019.2895286 (2019).

27. Wu, H., Yi, B., Ding, F., Feng, G. & Zhang, X. Linguistic steganalysis with graph neural networks. *IEEE Signal Process. Lett.* **28**, 558–562, DOI: 10.1109/LSP.2021.3062233 (2021).

28. Wang, H., Yang, Z., Yang, J., Chen, C. & Huang, Y. Linguistic steganalysis in few-shot scenario. *IEEE Transactions on Inf. Forensics Secur.* **18**, 4870–4882, DOI: 10.1109/TIFS.2023.3298210 (2023).

29. Gehrmann, S., Strobelt, H. & Rush, A. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 111–116, DOI: 10.18653/v1/P19-3019 (Association for Computational Linguistics, 2019).

30. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. & Finn, C. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 24950–24962 (PMLR, 2023).

31. Sadasivan, V. S., Kumar, A., Balasubramanian, S., Wang, W. & Feizi, S. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Mach. Learn. Res.* (2025).

32. Motwani, S. R. *et al.* Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems*, vol. 37, 73439–73486, DOI: 10.52202/079017-2336 (Curran Associates, Inc., 2024).

33. Zolkowski, A., Nishimura-Gasparian, K., McCarthy, R., Zimmermann, R. S. & Lindner, D. Early signs of steganographic capabilities in frontier LLMs. In *The Fourteenth International Conference on Learning Representations* (2026).

34. National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, National Institute of Standards and Technology (2015). DOI: 10.6028/NIST.FIPS.180-4.

35. Josefsson, S. The base16, base32, and base64 data encodings. RFC 4648, DOI: 10.17487/RFC4648 (2006).

36. Davis, K., Peabody, B. & Leach, P. Universally unique IDentifiers (UUIDs). RFC 9562, DOI: 10.17487/RFC9562 (2024).

37. Sennrich, R., Haddow, B. & Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–1725, DOI: 10.18653/v1/P16-1162 (Association for Computational Linguistics, 2016).

38. Kudo, T. & Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71, DOI: 10.18653/v1/D18-2012 (Association for Computational Linguistics, 2018).

39. Yan, R. & Murawaki, Y. Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 7076–7098, DOI: 10.18653/v1/2025.emnlp-main.361 (Association for Computational Linguistics, 2025).

40. Krawczyk, H., Bellare, M. & Canetti, R. HMAC: Keyed-hashing for message authentication. RFC 2104, DOI: 10.17487/RFC2104 (1997).

41. Dworkin, M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, National Institute of Standards and Technology (2007). DOI: 10.6028/NIST.SP.800-38D.

42. Nir, Y. & Langley, A. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, DOI: 10.17487/RFC8439 (2018).

43. Josefsson, S. & Liusvaara, I. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, DOI: 10.17487/RFC8032 (2017).

44. Flesch, R. A new readability yardstick. *J. Appl. Psychol.* **32**, 221–233, DOI: 10.1037/h0057532 (1948).

45. Wilson, E. B. Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.* **22**, 209–212, DOI: 10.1080/01621459.1927.10502953 (1927).

46. Kim, Y. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1746–1751, DOI: 10.3115/v1/D14-1181 (Association for Computational Linguistics, 2014).

47. He, P., Gao, J. & Chen, W. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations* (2023).

48. Efron, B. Bootstrap methods: Another look at the jackknife. *The Annals Stat.* **7**, 1–26, DOI: 10.1214/aos/1176344552 (1979).

49. Bates, D., Mächler, M., Bolker, B. & Walker, S. Fitting linear mixed-effects models using lme4. *J. Stat. Softw.* **67**, 1–48, DOI: 10.18637/jss.v067.i01 (2015).

50. Brooks, M. E. *et al.* glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R J.* **9**, 378–400, DOI: 10.32614/RJ-2017-066 (2017).

Acknowledgements

NA

Author contributions

A.V.M. alone worked on this paper and project.

Funding

NA

Competing interests

The author declares no competing interests.